
Midterm Exam 2 — Solutions

This document contains the official solutions for Midterm Exam 2 (April 16, 2026). Filled bubbles (●) indicate the correct answer for each multiple-choice question. Full worked solutions are provided for Parts 2–4.

Part	Topic	Points
Part 1	Conceptual (10 MCQ, 5 pts each)	50
Part 2	Short Numerical (3 questions, 5 pts each)	15
Part 3	Routing	20
Part 4	TCP trace	15
Total		100

Part 1 — Conceptual Questions**[50 pts, 5 pts each]**

Correct answers are marked with a filled bubble (●).

Q1. A router's forwarding table contains:

Prefix	Interface
10.0.0.0/8	eth0
10.20.0.0/16	eth1
10.20.5.0/24	eth2
0.0.0.0/0	eth3

A datagram arrives destined for **10.20.5.130**. Which interface does the router select?

- ☐ eth3
- ☐ eth0
- ☐ eth1
- ☒ eth2
- ☐ None of the above

Reasoning: 10.20.5.130 matches all four entries (/8, /16, /24, and default); longest-prefix match selects the most specific entry, which is the /24 → eth2.

Q2. An ISP is assigned the block **200.20.0.0/20** and subdivides it among customers using /24 blocks. Which statement is TRUE?

- ☐ The ISP must advertise a separate /24 route upstream for each customer.
- ☒ The ISP can advertise the single prefix 200.20.0.0/20 upstream, hiding its internal structure entirely.
- ☐ Upstream routers automatically discover the /24 subnets via OSPF flooding.
- ☐ The /20 block contains exactly 8 non-overlapping /24 subnets.
- ☐ None of the above.

Reasoning: CIDR route aggregation lets the ISP announce one summary prefix to the global table; upstream routers need not know the internal /24 structure (the /20 → /24 split yields $2^4 = 16$ subnets, not 8).

Q3. In IPv4, the TTL field:

- ☒ Is decremented at each router hop; when it reaches zero the datagram is dropped and the source receives an ICMP Time Exceeded message.
- ☐ Sets the maximum time in seconds a datagram may remain in the network.
- ☐ Is decremented by the sender for each retransmission, preventing lost datagrams from looping.
- ☐ Is used only for multicast traffic; unicast datagrams leave TTL unchanged.
- ☐ None of the above.

Reasoning: TTL is a hop count (not a timer); each router decrements it by one, and when it hits zero the datagram is discarded and ICMP Type 11 (Time Exceeded) is sent to the source to break routing loops.

Q4. IPv4 provides best-effort delivery — no guarantees of delivery, ordering, or timing. This is a deliberate design choice because:

- ☐ Modern fiber links have such low error rates that reliability guarantees are unnecessary.
- ☐ Best-effort is a temporary limitation of IPv4; IPv6 adds guaranteed delivery for real-time traffic.
- ☐ The engineers who designed IPv4 lacked the technology to implement reliability in 1981.
- ☒ Keeping routers simple and stateless allows the internet to scale to billions of devices across heterogeneous technologies, while letting endpoints implement exactly the reliability they need.
- ☐ None of the above.

Reasoning: This is the end-to-end argument: reliable network layers (like X.25) existed in 1981 but were deliberately rejected; pushing complexity to endpoints keeps routers fast, simple, and scalable across heterogeneous link technologies.

- Q5.** A web browser opens a TCP connection to a web server at port 443. Which statement about port numbers is TRUE?
- ☐ The browser uses port 443 as both source and destination port.
 - ☐ Port 443 is reserved exclusively for this connection until it closes; no other client can connect to port 443 meanwhile.
 - ☒ The OS assigns the browser a temporary, high-numbered source port; the server listens on well-known port 443.
 - ☐ Port numbers are negotiated during the three-way handshake; the server may reject the client's proposed port.
 - ☐ None of the above.

Reasoning: The OS selects a temporary, high-numbered source port for the client; many clients can connect to port 443 simultaneously because each connection is uniquely identified by the 4-tuple (src IP, src port, dst IP, dst port).

- Q6.** IPv4 uses 32-bit addresses, providing roughly 4.3 billion unique addresses. Yet the internet has far more than 4.3 billion connected devices. Which statement best explains how this works in practice?
- ☐ IANA reserves half the address space for future allocation, releasing new blocks as older ones fill up.
 - ☒ NAT allows many devices to share a single public IP address, so millions of devices can communicate using far fewer routable addresses than there are devices.
 - ☐ ISPs silently assign the same public IP to multiple customers, using MAC addresses to route return traffic to the correct host.
 - ☐ IPv6 has replaced IPv4 for all new connections since 2012, so address exhaustion no longer affects real deployments.
 - ☐ None of the above.

Reasoning: NAT translates between a single public IP and many private addresses using port numbers; the NAT router maintains a translation table to map incoming replies to the correct internal host.

- Q7.** When a link fails in a **distance-vector** network, convergence can be very slow. The root cause is:
- ☐ Distance-vector routers must rerun Dijkstra after every topology change, which is computationally expensive.
 - ☒ Routers can form a forwarding loop — each believing the other has a valid path to the lost destination — causing advertised costs to increment without bound.
 - ☐ Distance-vector requires a central coordinator to resolve conflicting route advertisements.
 - ☐ The failed router stops sending hello packets, so neighbors must wait for a long keepalive timeout before detecting the failure.
 - ☐ None of the above.

Reasoning: Count-to-infinity: two neighboring routers can each believe the other provides a route to the unreachable destination, forming a loop and incrementing the advertised cost by 1 each exchange round — DV uses Bellman-Ford, not Dijkstra.

- Q8.** An application developer chooses UDP over TCP for a live video streaming application. The best justification is:
- ☐ UDP is more secure — the lack of a handshake prevents man-in-the-middle attacks.
 - ☐ UDP automatically reduces its send rate when it detects packet loss, preventing congestion.
 - ☒ A retransmitted video frame arriving late is useless; it is better to skip it and display the next frame than to delay playback waiting for a retransmission.
 - ☐ UDP connections are stateful, so the OS can prioritize their traffic over TCP connections.
 - ☐ None of the above.

Reasoning: For real-time media, timeliness beats completeness: TCP's retransmission causes head-of-line blocking that delays newer frames, while UDP lets the application discard stale data and continue with the live stream.

- Q9.** TCP computes $RTO = SRTT + 4 \times RTTVAR$, where $RTTVAR$ tracks the variation in RTT samples. Why does the formula add a variance term rather than just using $SRTT$?
- ☐ The RFC mandates a minimum RTO of 1 second; the variance term ensures this floor is always met.
 - ☐ $SRTT$ grows monotonically over time; the variance term corrects for this drift.
 - ☒ On high-jitter paths, RTT samples can exceed $SRTT$ significantly; scaling the timeout to the actual spread prevents spurious retransmits without waiting unnecessarily long on stable paths.
 - ☐ The variance term accounts for asymmetric paths where the forward and return delays differ.
 - ☐ None of the above.

Reasoning: $RTTVAR$ estimates RTT variation; multiplying by 4 ensures the RTO covers nearly all legitimate RTT samples even on jittery paths, while on stable low-jitter paths $RTTVAR \approx 0$ so the RTO closely tracks $SRTT$ without unnecessary inflation.

Q10. TCP CUBIC replaces Reno’s linear window growth with a cubic function of time since the last congestion event. Which best describes why the shape is well-suited for high-bandwidth links?

- ☐ CUBIC uses multiplicative increase throughout, reaching high bandwidth faster than Reno on any link.
- ☒ CUBIC grows quickly when far below the previous peak, slows to probe cautiously near that peak, then accelerates again to search for newly available bandwidth — spending less time in the slow linear region that limits Reno on high-BDP paths.
- ☐ CUBIC eliminates slow start, beginning directly in congestion avoidance from the last known window size.
- ☐ CUBIC’s growth rate depends on the number of competing flows, making it inherently fairer than Reno.
- ☐ None of the above.

Reasoning: The cubic function’s three phases — fast ramp below $W_{\max}$, cautious plateau near $W_{\max}$, and renewed growth above — let CUBIC recover bandwidth quickly after a congestion event and then probe aggressively, eliminating the slow linear climb that bottlenecks Reno on high-bandwidth-delay-product paths.

Part 2 — Short Numerical**[15 pts, 5 pts each]**

Q11. *[5 pts]* A TCP sender has a window of **3,000 packets**, each 1500 bytes. The RTT is **100 ms**. What is the maximum throughput of this connection in **Mb/s**?

$$\text{Throughput} = \frac{\text{window} \times \text{MSS} \times 8}{\text{RTT}} = \frac{3,000 \times 1,500 \times 8}{0.1 \text{ s}} = \frac{36,000,000}{0.1} = 360,000,000 \text{ b/s}$$

Answer:

360

 Mb/s

Q12. *[5 pts]* A sender uses **stop-and-wait** (window = 1 packet) over a **1 Gb/s** link with **RTT = 100 ms**. Each packet is **1500 bytes**. What is the link utilization (%)?

$$t_{\text{tx}} = \frac{1500 \times 8 \text{ bits}}{10^9 \text{ b/s}} = 12 \mu\text{s} \quad \text{Utilization} = \frac{t_{\text{tx}}}{\text{RTT}} = \frac{12 \mu\text{s}}{100,000 \mu\text{s}} = 0.00012 = \mathbf{0.012\%}$$

The transmission time ($12 \mu\text{s}$) is tiny compared to the RTT (100 ms); the link sits idle nearly the entire time.

Answer:

0.012

 %

Q13. *[5 pts]* A **/26** subnet: how many **usable host addresses** does it contain?

Host bits = $32 - 26 = 6$. Total addresses = $2^6 = 64$. Usable = $64 - 2 = \mathbf{62}$ (subtract network address and broadcast).

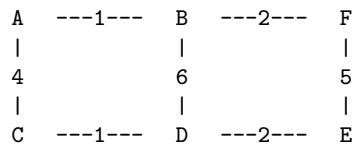
Answer:

62

Part 3 — Routing**[20 pts]**

The following undirected network has six nodes. All edge costs are shown.

Edge list: A-B:1 A-C:4 B-D:6 B-F:2 C-D:1 D-E:2 E-F:5

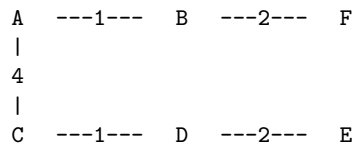


Q14. [4 pts] In what order does Dijkstra's algorithm finalize nodes, starting from **A**?

Trace: A(0) → tentative B=1, C=4. Finalize **B**(1) → update F=3, D=7. Finalize **F**(3) → update E=8. Finalize **C**(4) → update D=min(7,5)=5. Finalize **D**(5) → update E=min(8,7)=7. Finalize **E**(7).

Finalized	1st	2nd	3rd	4th	5th	6th
Node	A	B	F	C	D	E
Distance from A	0	1	3	4	5	7

Q15. [6 pts] Shortest-path spanning tree rooted at A (tree edges: A-B:1, A-C:4, B-F:2, C-D:1, D-E:2):



Path costs from A: $d(B) = 1$, $d(C) = 4$, $d(D) = 5$, $d(E) = 7$, $d(F) = 3$.

Q16. [3 pts] Forwarding table for router A (next-hop only):

Dest.	Next hop
B	B
C	C
D	C
E	C
F	B

D and E both route via C
(A→C→D and A→C→D→E).

Q17a. [3 pts] Distance vector of node **F**:

X	A	B	C	D	E	F
$d(F, X)$	3	2	7	7	5	0

F→B:2 (direct); F→A:3 (via B);

F→E:5 (direct); F→D:7 (via E or B);

F→C:7 (F→B→A→C = 2+1+4).

Q17b. [2 pts] E-F link changes from 5 to 1. Three entries in F's vector change:

$d(F, E)$: 5 → **1** (direct link now costs 1)

$d(F, D)$: 7 → **3** (F→E→D = 1 + 2)

$d(F, C)$: 7 → **4** (F→E→D→C = 1 + 2 + 1)

$d(F, A) = 3$ and $d(F, B) = 2$ are unchanged (the path via B is still shorter).

Q17c. [2 pts] **3 rounds.** Good news propagates one hop per round. Node **C** is the last to learn: the shortest updated path is F→E→D→C (3 hops from F). Round 1: E learns $d(E, F) = 1$. Round 2: D learns $d(D, F) = 3$. Round 3: C learns $d(C, F) = 4$.

Part 4 — TCP**[15 pts]**

Host A (192.168.1.10) is the client; Host B (10.50.4.22) is the server. The `tshark` capture below records their exchange.

No.	Time(s)	Src	Dst	Info	Seq=	Ack=	Len=	MSS=
1	0.000000	192.168.1.10	10.50.4.22	[SYN]	Seq=0		Len=0	MSS=1460
2	0.040000	10.50.4.22	192.168.1.10	[SYN,ACK]	Seq=0	Ack=1	Len=0	MSS=1460
3	0.040500	192.168.1.10	10.50.4.22	[ACK]	Seq=1	Ack=1	Len=0	
4	0.041000	192.168.1.10	10.50.4.22	[PSH,ACK]	Seq=1	Ack=1	Len=150	
5	0.081000	10.50.4.22	192.168.1.10	[ACK]	Seq=1	Ack=151	Len=0	
6	0.082000	10.50.4.22	192.168.1.10	[PSH,ACK]	Seq=1	Ack=151	Len=400	
7	0.082500	192.168.1.10	10.50.4.22	[FIN,ACK]	Seq=151	Ack=401	Len=0	
8	0.082600	10.50.4.22	192.168.1.10	[ACK]	Seq=401	Ack=152	Len=0	
9	0.120000	10.50.4.22	192.168.1.10	[FIN,ACK]	Seq=401	Ack=152	Len=0	
10	0.160000	192.168.1.10	10.50.4.22	[ACK]	Seq=152	Ack=402	Len=0	

Q18. [3 pts] Which packet numbers are associated with connection **establishment**? Which with **termination**? Which host initiated the close?

Establishment: 1, 2, 3 Termination: 7, 8, 9, 10 Close initiated by: Host A (sent FIN in packet 7)

Q19. [3 pts] Estimate the RTT for this connection. Show your reasoning.

Method 1 (handshake): SYN sent at $t = 0.000$ s; SYN-ACK received at $t = 0.040$ s. $RTT = 0.040 - 0.000 = 40$ ms.

Method 2 (data): PSH-ACK sent at $t = 0.041$ s; ACK received at $t = 0.081$ s. $\Delta = 40$ ms. (Both measurements agree.)

Answer: 40 ms

Q20. [6 pts] Suppose A and B run a large transfer over the same path. The slowest link in the round-trip path between A and B has a rate of **60 Mb/s**; RTT from Q19 = 40 ms; MSS = 1500 bytes.

(a) [2 pts] Bandwidth-delay product:

$$BDP = 60 \times 10^6 \text{ b/s} \times 0.040 \text{ s} = 2,400,000 \text{ bits} = \mathbf{300,000} \text{ bytes}$$

Answer: 300,000 bytes

(b) [2 pts] Minimum window to fill the pipe:

$$cwnd_{\min} = \frac{BDP}{MSS} = \frac{300,000}{1,500} = \mathbf{200} \text{ MSS}$$

Answer: 200 MSS

(c) [2 pts] The unscaled TCP window field is 16 bits, giving a maximum of $2^{16} - 1 = 65,535$ bytes. Since $65,535 < 300,000$, **window scaling is required**.

Find the minimum shift s such that $65,535 \times 2^s \geq 300,000$:

- $s = 2$: $65,535 \times 4 = 262,140 < 300,000$ (insufficient)
- $s = 3$: $65,535 \times 8 = 524,280 \geq 300,000$ (sufficient)

Minimum window scale factor: 3 (multiplier = 8; effective max window = 524,280 bytes).

Q21. [*3 pts*] The trace shows Seq = 0 for both A and B in packets 1 and 2. Are these the actual sequence numbers used on the wire? Explain why TCP uses the sequence numbers it does.

No — `tshark` displays *relative* sequence numbers, normalized to 0 for each direction, for readability. The actual Initial Sequence Numbers (ISNs) are **randomly chosen 32-bit values**. TCP uses random ISNs for two reasons:

1. **Stale segment protection:** prevents segments from a *previous* connection on the same port from being accepted as valid data in a new connection.
2. **Spoofing resistance:** an off-path attacker must guess the correct ISN to inject a forged segment; the full 32-bit random space makes this impractical.

— *End of Solutions* —